

2025-2026 学年面向对象程序设计模拟试卷 02-参考答案

一、选择题

1.

答案: B

在模板中, `std::vector<T>::size_type` 这样的名字依赖于模板参数 T, 编译器一开始无法确定它到底是不是一个类型名, 因此需要显式加上 `typename`。

所以正确写法是:

```
typedef typename std::vector<T>::size_type vec_sz;
```

这类题虽然偏语法, 但近年题目里确实很容易拿来卡人。

2.

答案: C

`operator[]` 必须写成成员函数。

类似地, 赋值运算符 `operator=`、函数调用运算符 `operator()` 和成员访问运算符 `operator->` 也必须是成员函数。

而 `operator+`、`operator<<`、`operator<` 都可以写成非成员函数。

3.

答案: D

`Num` 的构造函数被声明成了 `explicit`, 因此不能再进行从 `int` 到 `Num` 的隐式转换。

所以:

```
Num d = 7;
```

不能通过编译。

而 `Num b(3);` 和 `Num c = Num(5);` 都是显式构造, 因此仍然合法。

4.

答案: C

通过基类引用或指针调用虚函数时, 只要该函数是虚函数, 就会根据对象的**动态类型**进行绑定, 而不是看引用变量在定义时是什么静态类型。

所以 C 错。

其余选项都对:

1. 含纯虚函数的类一定是抽象类
2. 抽象类不能直接定义对象, 但可以定义指针和引用

3. 抽象类中可以有普通数据成员、构造函数和普通成员函数

5.

答案: C

`copy(v.begin(), v.end(), back_inserter(lst))` 是完全合法的, 表示把 `vector` 中的元素依次追加到 `list` 末尾。

其余选项错在:

- A: `list` 不支持下标访问
- B: `std::sort` 不能直接用于 `list`
- D: `remove_if` 只是重排元素并返回逻辑新结尾, 不会真正缩短容器长度, 通常还要再配合 `erase`

6.

答案: C

资源类如果只是简单共享底层裸指针, 而没有明确的共享管理机制, 就容易出现多个对象在析构时重复释放同一块内存的问题。

因此 C 正确。

A 错, 因为直接复制指针地址通常是不安全的。

B 错, 因为自我赋值 `x = x` 是完全可能发生的。

D 错, 因为赋值运算和复制构造虽然都涉及“复制”, 但发生时机和处理方式并不一样。

7.

答案: C

`unique_ptr` 不能复制, 只能移动。因此要把 `p` 管理的资源转交给 `q`, 必须写成:

```
std::unique_ptr<int> q = std::move(p);
```

移动之后, `p` 失去对原资源的拥有权。

8.

答案: A

把派生类对象按值赋给基类对象变量时, 只会保留基类子对象部分, 派生类额外的那一部分会被切掉, 这就是对象切片。

其余选项都错:

- B: 基类析构函数不是虚函数, 不代表派生类不能定义析构函数
- C: 对象切片恰恰常发生在按值传递和按值保存中
- D: 只有虚函数通过基类指针或引用调用时, 才会发生动态绑定

二、程序阅读题

1.

输出结果：

+N +0 *M /M -0 -N

分析如下：

1. new Derived('M') 时，先构造基类部分 Base(c + 1)，即 Base('N')，输出 +N
2. 再构造成员对象 member(c + 2)，即 Base('O')，输出 +0
3. 最后执行派生类构造函数体，输出 *M
4. delete p 时，因为基类析构函数是虚函数，所以会先调用派生类析构函数，输出 /M
5. 然后按成员逆序析构成员对象 member，输出 -0
6. 最后析构基类子对象，输出 -N

这题主要考察构造顺序、析构顺序和虚析构函数。

2.

输出结果：

1 6 6

c 初值为 1。

1. c++ 调用后置自增。它会先保存旧值，再把对象本身加 3，所以第一处输出的是旧值 1，而此后 c 变成 4
2. ++c 调用前置自增，直接把当前对象加 2，因此 c 从 4 变成 6，第二处输出 6
3. 最后再输出 c，仍然是 6

所以结果为 1 6 6。

3.

输出结果：

30

模板函数 sum_square 把区间中的每个元素平方后求和。

链表 nums 中依次存的是 1、2、3、4，因此：

$$1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$$

这题说明：只要算法只依赖迭代器的顺序访问能力，它就既可以用于 vector，也可以用于 list。

4.

输出结果：

A1 B1

这里同时考察了对象切片和“虚函数 / 非虚函数”的差别。

1. `A a = b;` 发生对象切片, 所以 `a` 已经是一个独立的 `A` 对象。
因此 `a.name()` 返回 `"A"`, `a.value()` 返回 `1`
2. `A& r = b;` 是基类引用绑定到派生类对象。
`name()` 是虚函数, 因此 `r.name()` 动态绑定到 `B::name()`, 得到 `"B"`
但 `value()` 不是虚函数, 因此 `r.value()` 仍按静态类型调用 `A::value()`, 结果是 `1`

所以最终输出 `A1 B1`。

三、程序填空题

1.

填空答案:

- (1) `typename std::vector<T>::size_type`
- (2) `std::domain_error("median of an empty vector")`
- (3) `v.begin(), v.end()`
- (4) `(v[mid] + v[mid - 1]) / 2`

说明如下:

1. 嵌套依赖类型前必须加 `typename`
2. 空向量没有中值, 应抛出异常
3. 中值函数通常要先对副本 `v` 排序
4. 若元素个数为偶数, 中值定义为中间两个元素的平均值

2.

填空答案:

- (5) `Str&`
- (6) `friend Str`
- (7) `ret += rhs`
- (8) `return os`

说明如下:

1. `operator+=` 应返回当前对象引用, 以支持链式写法
2. `operator+` 通常写成非成员友元函数, 内部先复制左操作数, 再调用 `+=`
3. 输出运算符最后必须返回对象本身

这是非常典型的一组运算符重载写法。

四、编程题

1. 模板函数 intersection_unique

这道题的关键在于两点：

1. 两个输入区间都已经有序，因此可以像归并那样双指针同时向前推进
2. 输出时要去重，也就是说即使某个公共元素在两个区间中都出现多次，最后也只写一次

一种直接而稳妥的写法如下：

```
template <class In1, class In2, class Out>
Out intersection_unique(In1 first1, In1 last1, In2 first2, In2 last2, Out dest)
{
    bool has_last = false;
    typename std::iterator_traits<In1>::value_type last_value =
        typename std::iterator_traits<In1>::value_type();

    while (first1 != last1 && first2 != last2) {
        if (*first1 < *first2) {
            ++first1;
        } else if (*first2 < *first1) {
            ++first2;
        } else {
            if (!has_last || !(*first1 == last_value)) {
                *dest = *first1;
                ++dest;
                last_value = *first1;
                has_last = true;
            }

            typename std::iterator_traits<In1>::value_type current = *first1;
            while (first1 != last1 && *first1 == current)
                ++first1;
            while (first2 != last2 && *first2 == current)
                ++first2;
        }
    }

    return dest;
}
```

思路解释如下：

1. 若 $*first1 < *first2$ ，说明当前第一个区间的值不可能出现在交集中，推进 $first1$
2. 若 $*first2 < *first1$ ，则推进 $first2$

3. 若两者相等，说明找到了公共元素
4. 为了去重，只在它和上一次输出值不同的情况下才写入目标位置
5. 一旦确认当前公共值是 x ，就要把两个区间里连续等于 x 的那一段都跳过去

对于测试数据：

a: 1 2 2 3 5 8 8 9

b: 0 2 2 4 5 8 10

公共元素是 2、5、8，并且每个只保留一次，所以输出正是：

2 5 8

2. Employee、Manager、Programmer 和 Company

这题本质上考察的是：

1. 抽象基类设计
2. 继承与多态
3. 基类指针容器
4. 通过 `clone()` 实现深拷贝
5. 复制构造、赋值运算符和析构函数的资源管理

其中最核心的点是：Company 保存的是 `Employee*`，但里面实际放的可能是 `Manager` 或 `Programmer`。因此在复制 `Company` 时，不能只复制指针地址，必须让每个对象通过虚函数 `clone()` 复制出自己的真实派生类型副本。

参考实现如下：

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Employee {
public:
    Employee(const string& n = "") : name(n) {}
    virtual ~Employee() {}

    virtual double income() const = 0;
    virtual void raise(double factor) = 0;
    virtual Employee* clone() const = 0;

    virtual void print(ostream& os) const {
        os << name << " " << income();
    }

protected:
```

```

    string name;
};

class Manager : public Employee {
public:
    Manager(const string& n = "", double s = 0, double b = 0)
        : Employee(n), salary(s), bonus(b) {}

    double income() const {
        return salary + bonus;
    }

    void raise(double factor) {
        salary *= factor;
    }

    Employee* clone() const {
        return new Manager(*this);
    }

private:
    double salary;
    double bonus;
};

class Programmer : public Employee {
public:
    Programmer(const string& n = "", double w = 0, int h = 0)
        : Employee(n), hourly_wage(w), hours(h) {}

    double income() const {
        return hourly_wage * hours;
    }

    void raise(double factor) {
        hourly_wage *= factor;
    }

    Employee* clone() const {
        return new Programmer(*this);
    }
}

```

```

private:
    double hourly_wage;
    int hours;
};

class Company {
public:
    Company() {}

    Company(const Company& rhs) {
        for (std::size_t i = 0; i != rhs.staff.size(); ++i)
            staff.push_back(rhs.staff[i]->clone());
    }

    Company& operator=(const Company& rhs) {
        if (this != &rhs) {
            clear();
            for (std::size_t i = 0; i != rhs.staff.size(); ++i)
                staff.push_back(rhs.staff[i]->clone());
        }
        return *this;
    }

    ~Company() {
        clear();
    }

    void add(const Employee& e) {
        staff.push_back(e.clone());
    }

    void raise_all(double factor) {
        for (std::size_t i = 0; i != staff.size(); ++i)
            staff[i]->raise(factor);
    }

    double total_income() const {
        double sum = 0;
        for (std::size_t i = 0; i != staff.size(); ++i)
            sum += staff[i]->income();
        return sum;
    }
}

```



```

friend ostream& operator<<(ostream& os, const Company& c) {
    for (std::size_t i = 0; i != c.staff.size(); ++i) {
        c.staff[i]->print(os);
        os << endl;
    }
    return os;
}

```

private:

```
vector<Employee*> staff;
```

```

void clear() {
    for (std::size_t i = 0; i != staff.size(); ++i)
        delete staff[i];
    staff.clear();
}

```

```
};
```

核对测试结果。最后逐行输出：

Wang 10000.00

Li 6600.00

与题目给出的结果一致。